

Java OOPs Interview Questions with Answers, Programs, Justifications (1-100)

✓ Part 1: Basics of OOPs (Q1 – Q10)

Q1. What is Object-Oriented Programming (OOP)? \ **Answer:** A paradigm based on objects that contain both data and behavior.

- **Concepts:** Class, Object, Inheritance, Polymorphism, Abstraction, Encapsulation
- **Real-world Example:** A car has properties (color, brand) and behaviors (drive, brake).

```
class Car {  
    String color = "Red";  
    void start() {  
        System.out.println("Car Started");  
    }  
}
```

Q2. What is a Class in Java? \ **Answer:** A blueprint or template that defines the variables and methods common to all objects of a type.

```
class Dog {  
    String breed;  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

Q3. What is an Object in Java? \ **Answer:** A runtime instance of a class.

```
Dog d = new Dog();  
d.bark();
```

Q4. What is Inheritance? \ **Answer:** Acquiring the properties and behaviors of a parent class.

```
class Animal {  
    void eat() { System.out.println("Eating..."); }  
}  
class Dog extends Animal {
```

```
void bark() { System.out.println("Barking..."); }  
}
```

Q5. What is Polymorphism? \ **Answer:** One name, many forms. Allows methods to behave differently based on context.

- **Compile-time (Overloading):**

```
class MathUtil {  
    int add(int a, int b) { return a + b; }  
    double add(double a, double b) { return a + b; }  
}
```

- **Runtime (Overriding):**

```
class Animal {  
    void sound() { System.out.println("Animal sound"); }  
}  
class Dog extends Animal {  
    void sound() { System.out.println("Bark"); }  
}
```

Q6. What is Abstraction? \ **Answer:** Hiding implementation details and showing only essential information.

```
abstract class Vehicle {  
    abstract void start();  
}  
class Bike extends Vehicle {  
    void start() {  
        System.out.println("Bike started");  
    }  
}
```

Q7. What is Encapsulation? \ **Answer:** Binding variables and methods into a single unit and restricting access using access modifiers.

```
class Student {  
    private int age;  
    public int getAge() { return age; }  
    public void setAge(int age) { this.age = age; }  
}
```

Q8. Difference between Abstraction and Encapsulation

Feature	Abstraction	Encapsulation
Purpose	Hides implementation details	Protects data and methods
Achieved by	Abstract class / Interface	Access modifiers and private fields

Q9. Can a class have multiple constructors? \ **Answer:** Yes. This is called constructor overloading.

```
class Book {
    Book() {
        System.out.println("Default Constructor");
    }
    Book(String title) {
        System.out.println("Book: " + title);
    }
}
```

Q10. What is Method Overloading? \ **Answer:** Same method name with different parameter lists.

```
class Calculator {
    int multiply(int a, int b) { return a * b; }
    double multiply(double a, double b) { return a * b; }
}
```

✓ Part 2: Tricky OOPs Questions (Q11 – Q20)

Q11. Can we create an object for an abstract class? \ **Answer:** No, abstract classes cannot be instantiated.

```
abstract class Shape {
    abstract void draw();
}
// Shape s = new Shape(); // ✗ Compilation error
```

Q12. Can an abstract class have constructors? \ **Answer:** Yes, constructors in abstract classes run when subclass objects are created.

```
abstract class Shape {
    Shape() {
        System.out.println("Shape constructor");
    }
}
class Circle extends Shape {
```

```

    Circle() {
        System.out.println("Circle constructor");
    }
}

```

Q13. Can a final method be overridden? Answer: No. Final methods cannot be overridden.

```

class A {
    final void show() {
        System.out.println("A");
    }
}
class B extends A {
    // void show() {} // ❌ Compilation error
}

```

Q14. Can we declare an interface method as static? Answer: Yes, since Java 8.

```

interface Utils {
    static void print() {
        System.out.println("Static method in interface");
    }
}

```

Q15. What happens if we write a return statement after throw? Answer: Compilation error: unreachable code.

```

void test() {
    throw new RuntimeException();
    // return; // ❌ Unreachable
}

```

Q16. Can constructors be inherited? Answer: No, constructors are not inherited by subclasses.

Q17. Can we use `` in constructor? Answer: Yes, it must be the first statement to call the parent constructor.

```

class Parent {
    Parent(String name) {
        System.out.println("Hello " + name);
    }
}
class Child extends Parent {

```

```
Child() {  
    super("Java");  
}  
}
```

Q18. Can we override a private method? \ Answer: No, private methods are not visible to subclasses.

Q19. Can we overload the `` method? \ Answer: Yes, but only `main(String[] args)` is used by JVM.

```
public class MainTest {  
    public static void main(String[] args) {  
        System.out.println("Main");  
    }  
    public static void main(int a) {  
        System.out.println("Overloaded main");  
    }  
}
```

Q20. Can abstract class have static methods? \ Answer: Yes. Abstract class can contain static methods.

```
abstract class MathUtils {  
    static int square(int x) {  
        return x * x;  
    }  
}
```

✅ Part 3: Advanced Tricky OOPs Questions (Q21 – Q40)

Q21. What is method overriding? \ Answer: Redefining a parent class method in a child class with the same signature.

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

Q22. What is dynamic method dispatch? \ **Answer:** When a superclass reference points to a subclass object and overridden methods are resolved at runtime.

```
Animal obj = new Dog();
obj.sound(); // Bark (runtime decision)
```

Q23. What is the difference between overloading and overriding?

Feature	Overloading	Overriding
Definition	Same method name, different params	Same method signature
Occurs in	Same class	Different classes (IS-A)
Runtime/Compile	Compile-time	Runtime

Q24. What is runtime polymorphism? \ **Answer:** It refers to method overriding where the call is resolved during runtime.

Q25. What is compile-time polymorphism? \ **Answer:** Method overloading; the compiler decides which method to call.

Q26. Can a method be both static and overridden? \ **Answer:** No. Static methods can be hidden but not overridden.

Q27. What is the use of instanceof operator? \ **Answer:** Checks if an object is an instance of a specific class/interface.

```
if (obj instanceof Dog) {
    System.out.println("It's a Dog");
}
```

Q28. What happens if superclass and subclass both have static method with same signature? \ **Answer:** Method hiding occurs. Which method is called depends on the reference type.

Q29. Can we override main method? \ **Answer:** No. We can overload but not override `main(String[] args)`.

Q30. What is covariant return type? \ **Answer:** When overriding a method, the return type can be a subclass of the original return type.

```
class A {
    A get() { return this; }
}
```

```
class B extends A {  
    B get() { return this; }  
}
```

Q31. What is the use of `` in constructors? Answer: It is used to call another constructor in the same class.

```
class Student {  
    int age;  
    Student() {  
        this(20); // calls parameterized constructor  
    }  
    Student(int age) {  
        this.age = age;  
    }  
}
```

Q32. What is constructor chaining? Answer: Calling one constructor from another using `this()` or `super()`.

Q33. Can a class be both final and abstract? Answer: No. Final classes cannot be extended, abstract ones must be.

Q34. Can we have a try block without catch or finally? Answer: No, it must be followed by either catch or finally.

Q35. Can an interface contain constructors? Answer: No. Interfaces cannot be instantiated.

Q36. Can abstract classes contain `` method? Answer: Yes.

```
abstract class Demo {  
    public static void main(String[] args) {  
        System.out.println("Main in abstract class");  
    }  
}
```

Q37. What if we define method as both ``? Answer: Compilation error. Static methods cannot be abstract.

Q38. Can we have an abstract method in a non-abstract class? Answer: No.

Q39. Can we overload a method by changing only return type? Answer: No. Return type alone is not sufficient to distinguish methods.

Q40. Can an interface extend a class? Answer: No. An interface can only extend another interface.

✓ Part 4: OOPs Concepts in Real-Time and Edge Cases (Q41 – Q60)

Q41. What is real-time use of abstraction? Answer: In payment systems, different payment modes implement the same method differently (e.g., `pay()`).

Q42. Can we create an instance of an interface using an anonymous class? Answer: Yes.

```
interface Greet {
    void sayHello();
}
class Test {
    public static void main(String[] args) {
        Greet g = new Greet() {
            public void sayHello() {
                System.out.println("Hello");
            }
        };
        g.sayHello();
    }
}
```

Q43. What is the difference between interface and abstract class?

Feature	Interface	Abstract Class
Multiple Inheritance	Supported	Not supported
Constructors	Not allowed	Allowed
Fields	public static final	Any modifier

Q44. Can we cast subclass object to superclass type? Answer: Yes, it's called upcasting.

```
Dog d = new Dog();
Animal a = d; // upcasting
```

Q45. Can we cast superclass to subclass directly? Answer: Only with explicit casting and valid instance.

```
Animal a = new Dog();
Dog d = (Dog) a; // Valid
```

Q46. What is method hiding? Answer: When a static method in subclass hides the same method in superclass.

Q47. What happens if subclass constructor does not call `super`? Answer: Java inserts a default `super()` call automatically if no other is provided.

Q48. What is the diamond problem? Why is it not in Java? Answer: Occurs in multiple inheritance of classes. Java avoids it using interfaces only.

Q49. How to access parent class method if overridden? Answer: Using `super.methodName()`

```
class Parent {  
    void show() { System.out.println("Parent"); }  
}  
class Child extends Parent {  
    void show() {  
        super.show();  
        System.out.println("Child");  
    }  
}
```

Q50. Can we have static class in Java? Answer: Only inner classes can be static.

```
class Outer {  
    static class Inner {  
        void show() {  
            System.out.println("Static Inner Class");  
        }  
    }  
}
```

Q51. Can interface contain default methods? Answer: Yes, since Java 8.

```
interface Drawable {  
    default void draw() {  
        System.out.println("Drawing");  
    }  
}
```

Q52. Can constructor be `final`? Answer: Yes. Used in Singleton pattern.

```
class Singleton {  
    private Singleton() {}  
}
```

Q53. Can we call constructor explicitly? Answer: No. Constructor is called implicitly via `new`.

Q54. What is `` operator? Answer: Used to test whether an object is an instance of a class/interface.

```
if(obj instanceof String) {...}
```

Q55. Can a method be both ``? Answer: No. Synchronization needs implementation.

Q56. What is tight coupling in OOP? Answer: When one class is highly dependent on another class.

Q57. What is loose coupling? Answer: Classes are independent and interact via interfaces.

Q58. Can static block throw exceptions? Answer: Only unchecked exceptions.

Q59. Can a final variable be changed in constructor? Answer: Yes, if not already assigned.

Q60. Can we override `` method? Answer: Yes, but it's deprecated in Java 9+

✓ Part 5: Object Class, Advanced Keywords, and Design (Q61 – Q80)

Q61. What are some methods of Object class? Answer: `toString()`, `equals()`, `hashCode()`, `clone()`, `finalize()`

Q62. What is the use of toString()? Answer: Returns string representation of an object.

```
public String toString() {  
    return "Object details";  
}
```

Q63. What does equals() method do? Answer: Compares the content of two objects.

```
public boolean equals(Object obj) {  
    return this.id == obj.id;  
}
```

Q64. What is hashCode()? Answer: Returns unique integer representing the object for hashing.

Q65. Can clone() be used without implementing Cloneable? Answer: No. Throws `CloneNotSupportedException`

Q66. What is the use of final keyword? Answer: Used to mark variable, method or class as non-changeable.

Q67. What is the use of static keyword? \ **Answer:** Belongs to class rather than instance.

Q68. What are static blocks used for? \ **Answer:** To initialize static data members.

Q69. What is the difference between == and equals()? \ Answer: `==` checks reference, `.equals()` checks content.

Q70. What is method overriding? \ **Answer:** Redefining a parent method in subclass with same signature.

Q71. Can we override a static method? \ **Answer:** No. It will be method hiding.

Q72. Can we override a final method? \ **Answer:** No. Final methods can't be overridden.

Q73. What is the use of super keyword? \ **Answer:** To call parent class constructor/method.

Q74. What is the use of this keyword? \ **Answer:** Refers to current class object.

Q75. Can we override a private method? \ **Answer:** No. It's not visible in subclass.

Q76. Can we overload main method? \ **Answer:** Yes. But JVM calls only `main(String[] args)`.

Q77. What is Singleton Design Pattern? \ **Answer:** Ensures one instance of class.

```
class Singleton {  
    private static Singleton instance = new Singleton();  
    private Singleton() {}  
    public static Singleton getInstance() {  
        return instance;  
    }  
}
```

Q78. What is Factory Design Pattern? \ **Answer:** Uses a method to create object based on condition.

Q79. What is immutable class in Java? \ **Answer:** Class whose objects cannot be changed once created (e.g., String).

Q80. Can we inherit constructor in subclass? \ **Answer:** No. Constructors are not inherited.

Part 6: Final Tricky Questions and Real-Time Scenarios (Q81 – Q100)

Q81. Can we make constructor final? \ **Answer:** No. Constructors cannot be final.

Q82. What is the real-time use of abstract class? \ **Answer:** For common base templates in frameworks (e.g., JDBC).

- Q83. What is composition in Java?** \ Answer: Has-A relationship. One class contains another.
- Q84. What is aggregation?** \ Answer: A weak Has-A relationship (e.g., Student has a Department).
- Q85. Can abstract class have both abstract and concrete methods?** \ Answer: Yes.
- Q86. What is a pure abstract class?** \ Answer: An interface - contains only abstract methods.
- Q87. What happens if we don't override abstract method in subclass?** \ Answer: Compilation error.
- Q88. Can we declare class as both abstract and final?** \ Answer: No. Contradictory rules.
- Q89. What is default access modifier?** \ Answer: No modifier = package-private.
- Q90. Can abstract class extend concrete class?** \ Answer: Yes.
- Q91. Can a class implement multiple interfaces?** \ Answer: Yes. Java supports multiple inheritance via interfaces.
- Q92. Can interface extend multiple interfaces?** \ Answer: Yes.
- Q93. What is Liskov Substitution Principle (LSP)?** \ Answer: Subclasses should be usable in place of parent class.
- Q94. What is Open Closed Principle (OCP)?** \ Answer: Classes should be open for extension, closed for modification.
- Q95. What is dependency injection?** \ Answer: Supplying external dependencies to a class.
- Q96. Difference between deep copy and shallow copy?** \ Answer: Deep copy copies all nested objects; shallow copies references.
- Q97. Can we use interface as a reference type?** \ Answer: Yes. Common in loose coupling.
- Q98. Real-world example of polymorphism?** \ Answer: Print method prints int, float, String, etc.
- Q99. Real-world use of encapsulation?** \ Answer: ATM hides internal logic; exposes limited methods.
- Q100. Can interface contain constructor or instance block?** \ Answer: No. Interface can't have constructor or instance block.

 **All 100 Questions Completed!**

Let me know if you'd like this converted into a downloadable PDF, or want a version with MCQs or visuals!